W04 Team Activity: Foundation Programs Design

Program #1 - YouTube Videos (Abstraction)

1) Classes (and responsibilities)

Minimum classes (from your team):

- Video: stores video data and its list of comments; can compute how many comments it has.
- Comment: stores the commenter name and the comment text.

Optional (helpful but not required):

- VideoLibrary / Program (orchestrator): creates the videos, adds comments, and prints output (often this logic lives in Program.cs, not necessarily as a separate class).

2) Methods (behaviors)

Comment

- Comment(string commenterName, string text) (constructor)
- GetCommenterName(): string (optional)
- GetText(): string (optional)
- ToString(): string → returns something like "Name: text" (optional but convenient)

Video

- Video(string title, string author, int lengthSeconds) (constructor)
- AddComment(Comment comment): void
- GetNumberOfComments(): int
- GetComments(): List<Comment> (or return a copy / read-only)
- GetDisplayText(): string (optional) → builds the line you print (title, author, length, #comments)

Program (main)

- Main(string[] args): void
- Create 3–4 videos
- Add 3–4 comments per video
- Loop and print all required output

3) Attributes (member variables)

Comment

- _commenterName: string
- _text: string

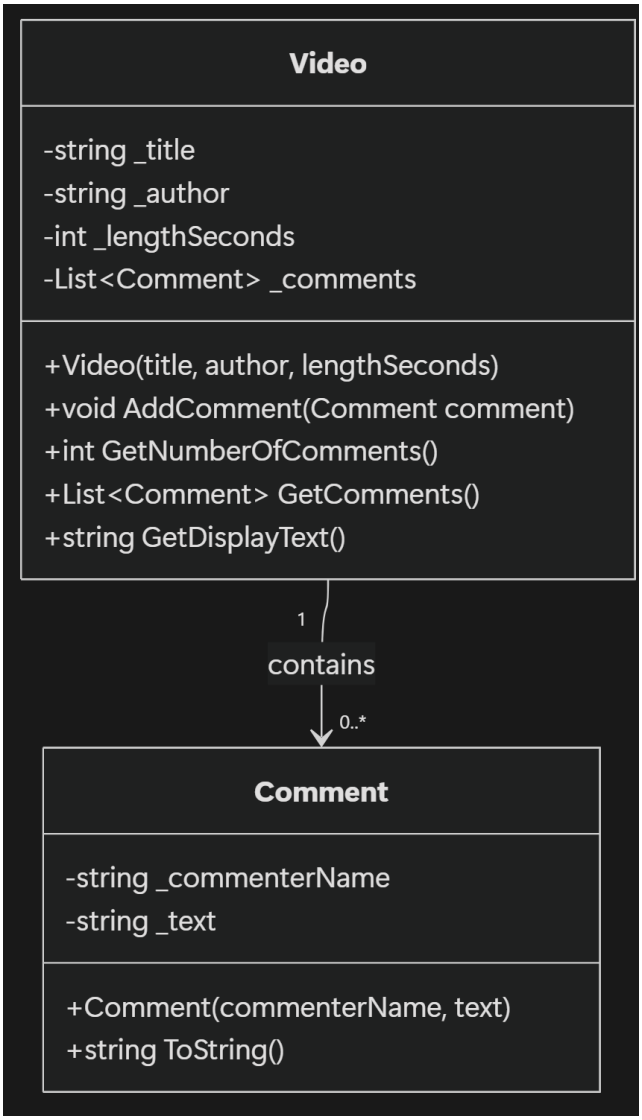
Video

- _title: string
- _author: string
- _lengthSeconds: int
- _comments: List<Comment>

4) Class diagram (simple UML)

```
classDiagram
class Video {
  -string _title
  -string _author
  -int _lengthSeconds
  -List~Comment~ _comments
  +Video(title, author, lengthSeconds)
  +void AddComment(Comment comment)
  +int GetNumberOfComments()
  +List~Comment~ GetComments()
  +string GetDisplayText()
}

class Comment {
  -string _commenterName
  -string _text
  +Comment(commenterName, text)
  +string ToString()
}
Video "1" --> "0..*" Comment : contains
```



5) Program flow (how it runs / how methods relate)

Mermaid

flowchart TD

A[Start Main] --> B[Create List<Video>]

B --> C[Create Video 1..4]

C --> D[For each Video: add 3..4 Comment]

D --> E[Loop through videos]

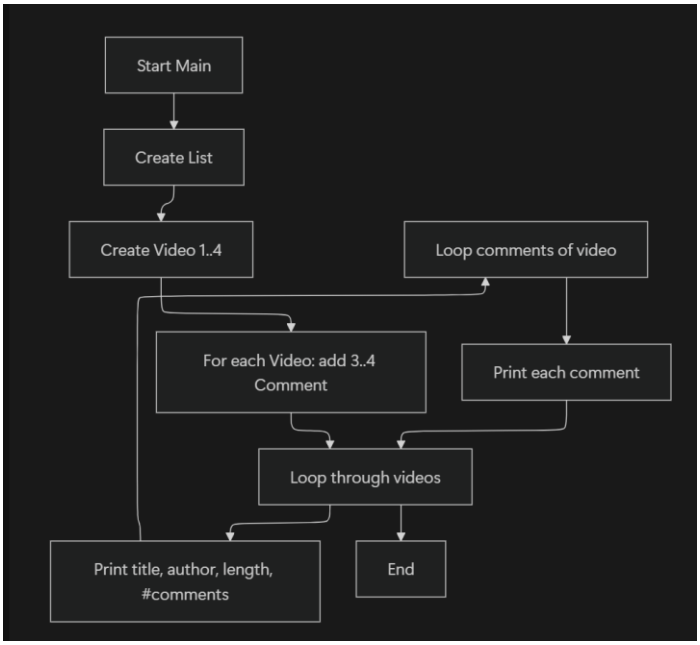
E --> F[Print title, author, length, #comments]

F --> G[Loop comments of video]

G --> H[Print each comment]

H --> E

E --> I[End]



Program #2 - Online Ordering (Encapsulation)

1) Classes (and responsibilities)

Required classes (from your team):

- **Product**: name, id, price, quantity; computes its total cost.
- **Address**: stores full address; knows if it's in the USA; returns formatted address string.
- **Customer**: name + address; knows whether they live in the USA (without checking the country directly).
- **Order**: list of products + customer; computes totals; generates packing and shipping labels.

Optional (only if you want to improve design):

- ReceiptPrinter / OrderService: separate printing from calculation (not required).

2) Methods (behaviors)

Product

- Product(string name, string productId, double price, int quantity)
- GetTotalCost(): double → price * quantity
- (Optional) getters like
- GetName(), GetProductId(), etc.

Address

- Address(string street, string city, string stateOrProvince, string country)
- IsInUSA(): bool
- GetFullAddress(): string → formatted with line breaks

Customer

- Customer(string name, Address address)

- LivesInUSA(): bool → calls address.IsInUSA()
- GetName(): string
- GetAddress(): Address **or** GetShippingAddressString(): string (choose one approach)

Order

- Order(Customer customer)
- AddProduct(Product product): void
- GetShippingCost(): double → 5 if USA, 35 otherwise
- GetTotalCost(): double → sum of products + shipping
- GetPackingLabel(): string → list products (name - id)
- GetShippingLabel(): string → customer name + full address

Program

(main)

- Main(string[] args): void
    - Create 2–3 orders with different customers (USA and non-USA)
    - Add products to each order
    - Print packing label, shipping label, and total cost for each order

3) Attributes (member variables)

Product

- _name: string
- _productId: string
- _price: double
- _quantity: int

Address

- _street: string
- _city: string
- _stateOrProvince: string
- _country: string

Customer

- _name: string
- _address: Address

Order

- _products: List<Product>
- _customer: Customer

4) Class diagram (simple UML)

classDiagram

```
class Product {
 -string _name
 -string _productId
 -double _price
 -int _quantity
 +Product(name, productId, price, quantity)
 +double GetTotalCost()
 +string GetName()
 +string GetProductId()
}
class Address {
 -string _street
 -string _city
 -string _stateOrProvince
 -string _country
 +Address(street, city, stateOrProvince, country)
 +bool IsInUSA()
 +string GetFullAddress()
}
class Customer {
 -string _name
 -Address _address
 +Customer(name, address)
 +bool LivesInUSA()
 +string GetName()
 +Address GetAddress()
}

class Order {
 -List~Product~ _products
```

-Customer _customer

+Order(customer)

+void AddProduct(Product product)

+double GetShippingCost()

+double GetTotalCost()

+string GetPackingLabel()

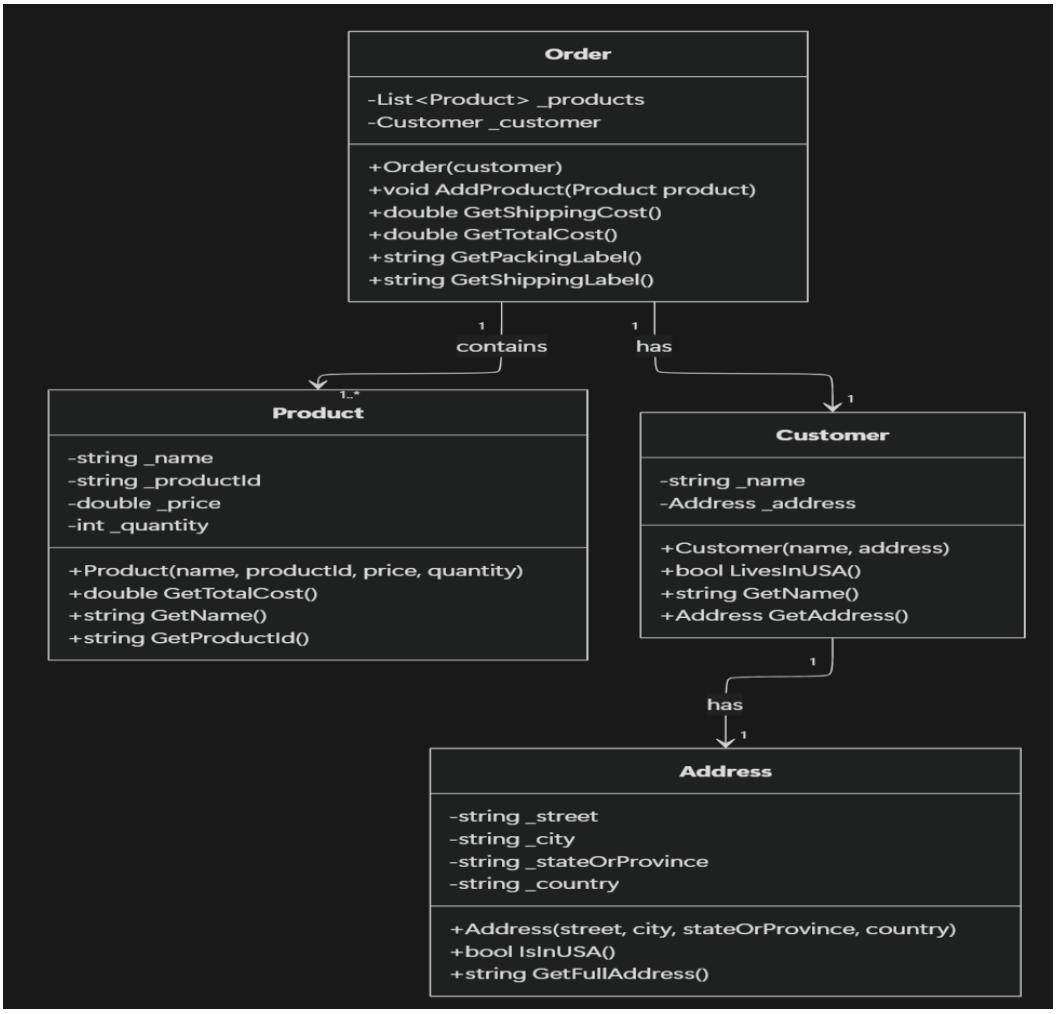
+string GetShippingLabel()

}

Order "1" --> "1" Customer : has

Customer "1" --> "1" Address : has

Order "1" --> "1..*" Product : contains



5) Program flow (how it runs / method relationships)

flowchart TD

```
A[Start Main] --> B[Create Customer + Address]

B --> C[Create Order(customer)]

C --> D[Create Products]

D --> E[order.AddProduct(product) ...]

E --> F[Print Packing Label]

F --> G[Order.GetPackingLabel()<br>loops products -> name + id]

E --> H[Print Shipping Label]

H --> I[Order.GetShippingLabel()<br>customer name + address.GetFullAddress()]

E --> J[Print Total Cost]

J --> K[Order.GetTotalCost()]

K --> L[Sum products: product.GetTotalCost()]

K --> M[Add shipping: Order.GetShippingCost()]

M --> N[Order uses customer.LivesInUSA() -> address.IsInUSA()]

N --> O[End]
```